

Operating System Exam Paper 2024/25 Solution

Question 1: a) What are the four conditions that are required for Deadlock to occur?



Add an image

- List

Four conditions required for a locked system

1. Mutual exclusion: allowing only one process access to dedicated resource

2. Resource holding and waiting: may hold a resource (not releasing the resource); while waiting assignment of another resource, held by another job (for other job to retreat)

3. No pre-emption: no resource can be forcibly removed from a process holding it

4. Circular wait: each process waiting for another to voluntarily release so at least one can continue

- All conditions required for deadlock
- Resolving deadlock: remove one of the condition

Question 2: b) Explain the code in your own words (10 marks)

```

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    pthread_t tid;          /* thread identifier */
    int retcode;

    if (argc != 4) {
        printf ("program exiting: incorrect number of command line arguments\n");
        exit(0);
    }

    /* Create Thread */
    retcode = pthread_create(&tid, NULL, my_thread, argv[3]);
    if (retcode != 0) {
        fprintf (stderr, "Unable to create thread\n");
        exit (1);
    }

    pthread_join(tid, NULL);

    printf("The value returned by the thread is %d", value);
    printf ("\nThe end of the program\n");

    pthread_exit(0);

    /* Plain that this thread does */
    my_thread(void *param)
    {
        int i = atoi(param);
        printf("I am the child thread passed the value %d \n", i);

        value = i*i*i*i;

        pthread_exit(0);
    }
}

```

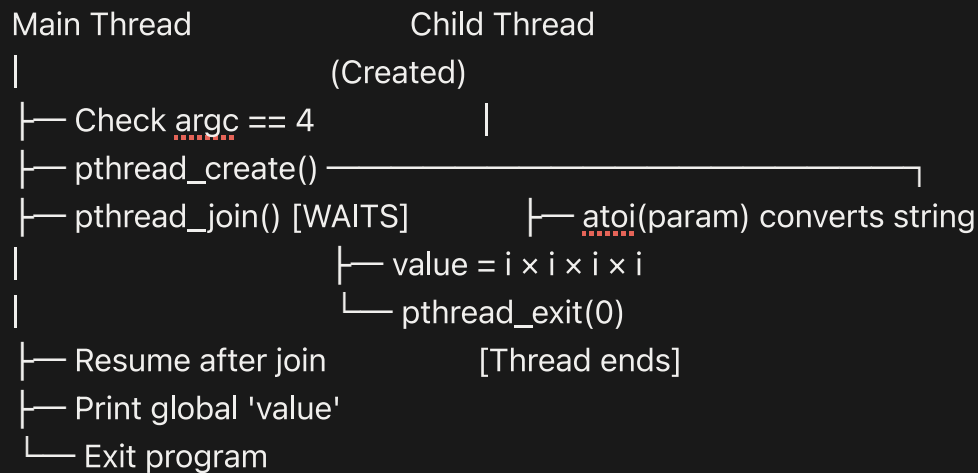
A. Program Overview

This C program demonstrates POSIX thread (pthread) usage to calculate the fourth power of a number provided as a command-line argument. The computation occurs in a separate thread while the main thread waits and displays the result.

B. Key Components

1. Global Variable: value - Shared between threads for result storage
2. Thread Function: my_thread() - Receives parameter, computes i^4 , stores in value
3. Main Function: Validates input, creates thread, waits for completion, outputs result
4. Thread Management: Uses pthread_create(), pthread_join(), pthread_exit()

C. Execution Flow with Diagram



D. Technical Details

1. Argument Passing: argv[3] (char) is implicitly cast to void for thread parameter
2. Synchronization: pthread_join() ensures sequential execution - no race condition on value
3. Data Conversion: String $\rightarrow$ int via atoi() in thread; result stored in shared global variable
4. Thread Termination: Both threads use pthread_exit(0) for clean termination

E. Critical Analysis & Limitations

Strengths:

- Simple demonstration of thread creation and communication
- Proper synchronization prevents data races
- Clear separation of concerns (main manages, thread computes)

Limitations:

1. Global variable usage is not scalable for complex programs
2. Better approach: Use pthread_join() to retrieve result directly:

```

c
void *result;
pthread_join(tid, &result);
  
```

3. No error checking for atoi() conversion failures
4. Old-style main() declaration should be int main(int argc, char *argv[])